

Modeling Lateral Error Correction in a Line-Following Robot Using Second-Order ODE's and Neural ODE's

Gabriel Melgarejo

Math 2A — Differential Equations

Foothill College

June 15, 2026

Abstract

We model the lateral error correction of a line-following robot using a second-order linear homogeneous ordinary differential equation derived from Newton's second law and PD control theory. By solving the initial value problem analytically using the characteristic equation method, we demonstrate how controller gains determine whether the robot oscillates, corrects smoothly, or returns sluggishly to the line. A Neural ODE is trained to verify the analytical result computationally.

Contents

1	Introduction	4
1.1	Brief History, and background knowledge	4
1.2	Research Question	4
1.3	Importance of the Research Question	4
2	Summary of ODE Techniques	5
3	Background Information	5
3.1	Key Terms and Definitions	5
3.2	How a Line-Following Robot Works	6
3.3	The Three Correction Behaviors	7
3.4	Underdamped Case	7
3.5	Critically Damped Case	8
3.6	Overdamped Case	9
4	Setting up the IVP	10
5	Derivation of the Mathematical Model (IVP)	11
5.1	Step 1 — Newton's Second Law	11
5.2	Step 2 — PD Controller Force	11
5.3	Step 3 — Combining Newton's Law with PD Control	12
5.4	Initial Conditions and Parameters	14
5.5	Assumptions Made	15
6	Solution to the IVP	16
6.1	Step 1 — Writing the Characteristic Equation	16
6.2	Step 2 — Calculating the Discriminant	16
6.3	Step 3 — Finding the Roots	17
6.4	Step 4 — Writing the General Solution	17
6.5	Step 5 — Applying Initial Conditions	18
6.6	Step 6 — Particular Solution	19
6.7	Graph of the Particular Solution	20
7	Prediction	21
7.1	Evaluating $e(2)$	21
7.2	Interpretation	21

7.3	Verification Using MATLAB	22
8	Limitations of the Model	23
8.1	Accuracy of the Strategy	23
8.2	Variables Not Accounted For	23
8.3	Criteria for Model Validity	23
9	Conclusion	24
9.1	Summary and Answer to Research Question	24
9.2	Unanswered Question for Future Students	24
10	Bonus: Neural ODE Verification	25
10.1	What is a Neural ODE?	25
10.2	Data Generation	25
10.3	Neural Network Architecture and Training	26
10.4	Results and Comparison	26
10.5	Prediction Verification	28
A	MATLAB Code	30
B	Python Code — Neural ODE	34

List of Figures

1	Underdamped: $e(t)$ vs. time. Robot oscillates across the line before settling.	7
2	Underdamped: top-view diagram of robot zigzagging across the line.	7
3	Critically damped: $e(t)$ vs. time. Fastest smooth return to the line.	8
4	Critically damped: top-view diagram of robot returning smoothly to the line.	8
5	Overdamped: $e(t)$ vs. time. Robot drifts back slowly without oscillating. . .	9
6	Overdamped: top-view diagram of robot slowly returning to the line.	9
7	Before PD correction: $e(0) = 0.1$ m	10
8	After PD correction: $e(t) \approx 0$ m	10
9	Particular solution $e(t)$ vs. time for the underdamped case. Parameters: $m =$ $1, k_p = 4, k_d = 2, e(0) = 0.1, e'(0) = 0$. Generated using MATLAB.	20
10	Particular solution $e(t)$ vs. time for the underdamped case with prediction at $t = 2$ seconds. At $t = 2, e(2) \approx -0.0153$ m, confirming the robot has crossed the line. Parameters: $m = 1, k_p = 4, k_d = 2, e(0) = 0.1, e'(0) = 0$. Generated using MATLAB.	22
11	Comparison of analytical ODE solution (green) vs. Neural ODE learned so- lution (black dashed) for robot lateral error correction. Both curves match closely, confirming the neural network successfully learned the correction dy- namics from data. Generated using Python and PyTorch in Visual Studio Code.	27
12	Neural ODE training results and prediction comparison. The model was trained on 1000 data points generated from the analytical solution of the underdamped line-following robot. After 5000 training iterations, the Neural ODE achieved a minimum loss of 3.6×10^{-7} and predicted the lateral error at $t = 2$ seconds with high accuracy.	27

1. Introduction

1.1. Brief History, and background knowledge

Interest in automatic error correction began in 1922 when Nicolas Minorsky introduced PID control for automatic ship steering for the U.S. Navy [1]. In 1942, Ziegler and Nichols developed systematic tuning methods, making PID and PD controllers standard across robotics and industry [2]. So, what is a PD controller? A PD (Proportional-Derivative) controller is a simplified version from PID (Proportional-Integral-Derivative) controller, which is a feedback control algorithm used in engineering whose function is to automatically correct errors between a system's current state and its desired target state in real time. It reads two things: how far off the target something is, and how fast that error is changing, then applies a corrective force to fix it [3].

Here are our PD and PID controller equations:

$$\text{PD: } u(t) = k_p \cdot e(t) + k_d \cdot e'(t) \quad [3]$$

$$\text{PID: } u(t) = k_p \cdot e(t) + k_i \cdot \int e(t) dt + k_d \cdot e'(t) \quad [3]$$

1.2. Research Question

Can the lateral error correction of a line-following robot under PD control be modeled as a second-order ODE, and if so, what does the solution predict about the robot's correction behavior, will it be oscillatory, smooth, or sluggish? (See Section 3 for definitions of each case.)

1.3. Importance of the Research Question

Error correction is fundamental to all robotic systems. Understanding the mathematics behind how robots correct themselves enables engineers to design more reliable and efficient systems. PD and PID controllers are used in real robotics applications including Amazon warehouse robots, Tesla lane-keeping systems, and industrial automation, representing over 95% of all industrial feedback control systems [4]. Mastering this mathematics is essential for any robotics, controls engineer.

2. Summary of ODE Techniques

This project applies the **Characteristic Equation Method** to solve a second-order linear homogeneous ODE with constant coefficients. The characteristic equation is used to determine the form of the general solution, after which initial conditions are applied to obtain the particular solution to understand the correction behavior. Finally we will use an example to see and predict how far from the line will the car be at a specific time. Initial conditions are applied to obtain the particular solution. As a bonus, a **Neural ODE**, a neural network that learns ODE dynamics from data, verifies the analytical result computationally [6].

3. Background Information

3.1. Key Terms and Definitions

The following terms are essential for understanding the mathematical model developed in this project.

- **Lateral Error $e(t)$:** The perpendicular distance between the robot and the line at time t , measured in meters. When $e(t) = 0$, the robot is perfectly on the line.
- **PD Controller (Proportional-Derivative):** A feedback control system that applies a corrective force based on two terms: the current error magnitude and its rate of change. It does not use past accumulated error (that would require a full PID controller, producing a third-order ODE beyond the scope of this course).
- **Proportional Gain k_p :** A controller parameter that determines how strongly the robot corrects based on the current error. A larger k_p produces a stronger correction force.
- **Derivative Gain k_d :** A controller parameter that determines how strongly the robot reacts to the rate of change of the error. A larger k_d slows the correction down to prevent overshoot.
- **Damping:** The tendency of a system to resist oscillation and return to equilibrium. In this project, damping is controlled by the derivative gain k_d .
- **Underdamped (Oscillatory):** When $k_d^2 - 4mk_p < 0$. The robot oscillates across the line before settling.
- **Critically Damped (Smooth):** When $k_d^2 - 4mk_p = 0$. The robot returns to the line as fast as possible without oscillating. This is the ideal case for robotics applications.

- **Overdamped (Sluggish):** When $k_d^2 - 4mk_p > 0$. The robot drifts back to the line slowly without oscillating but takes longer than the critically damped case.

3.2. How a Line-Following Robot Works

A line-following robot is an autonomous vehicle that uses sensors to detect a line on the ground and continuously corrects its position to stay on that line [5]. The process works as follows:

1. The sensor (camera or infrared) detects how far the robot is from the line — this is the lateral error $e(t)$.
2. The PD controller reads $e(t)$ and $e'(t)$ and computes a corrective steering force.
3. The corrective force is applied to the wheels, steering the robot back toward the line.
4. This process repeats continuously, forming a closed-loop feedback system.

3.3. The Three Correction Behaviors

The three correction behaviors defined in Section 3.1 are summarized in the table below, along with the discriminant condition that produces each one:

Case	Condition	Robot Behavior
Underdamped	$k_d^2 - 4mk_p < 0$	Robot zigzags across line
Critically damped	$k_d^2 - 4mk_p = 0$	Fastest smooth return (ideal)
Overdamped	$k_d^2 - 4mk_p > 0$	Robot drifts back slowly

3.4. Underdamped Case

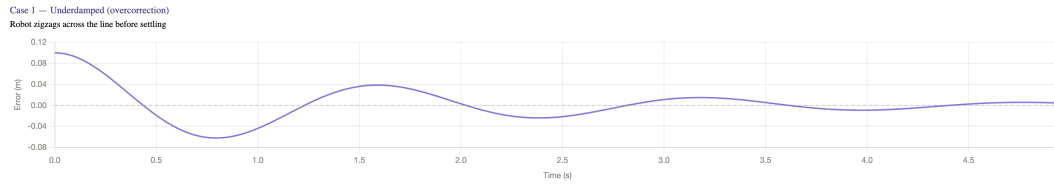


Figure 1: Underdamped: $e(t)$ vs. time. Robot oscillates across the line before settling.

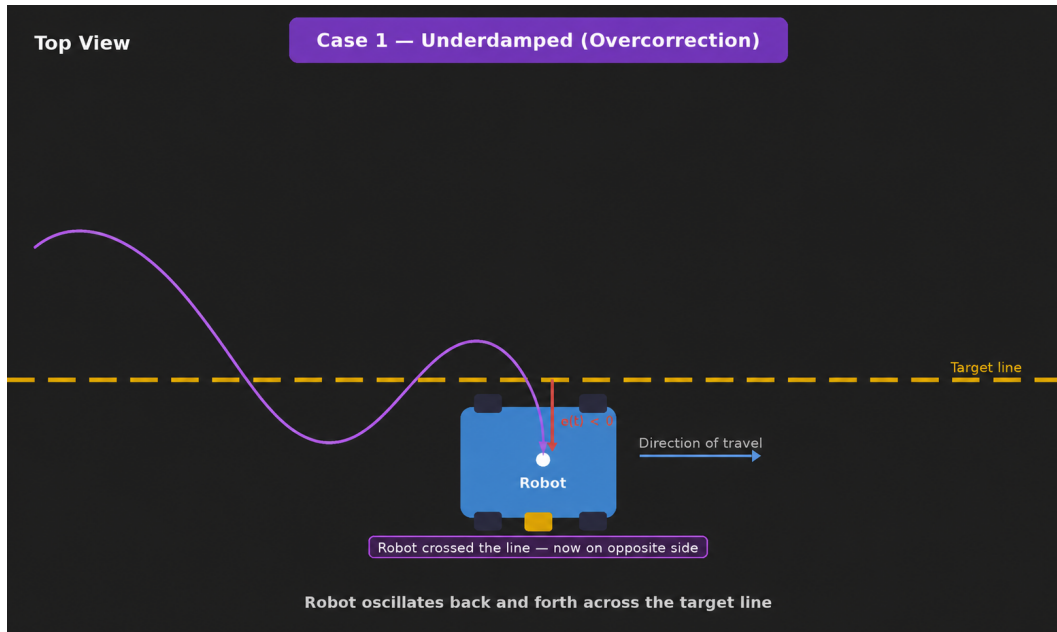


Figure 2: Underdamped: top-view diagram of robot zigzagging across the line.

3.5. Critically Damped Case

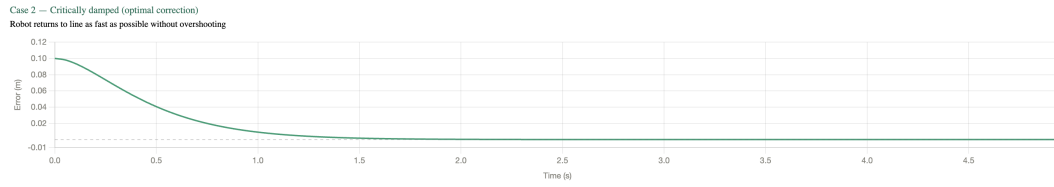


Figure 3: Critically damped: $e(t)$ vs. time. Fastest smooth return to the line.

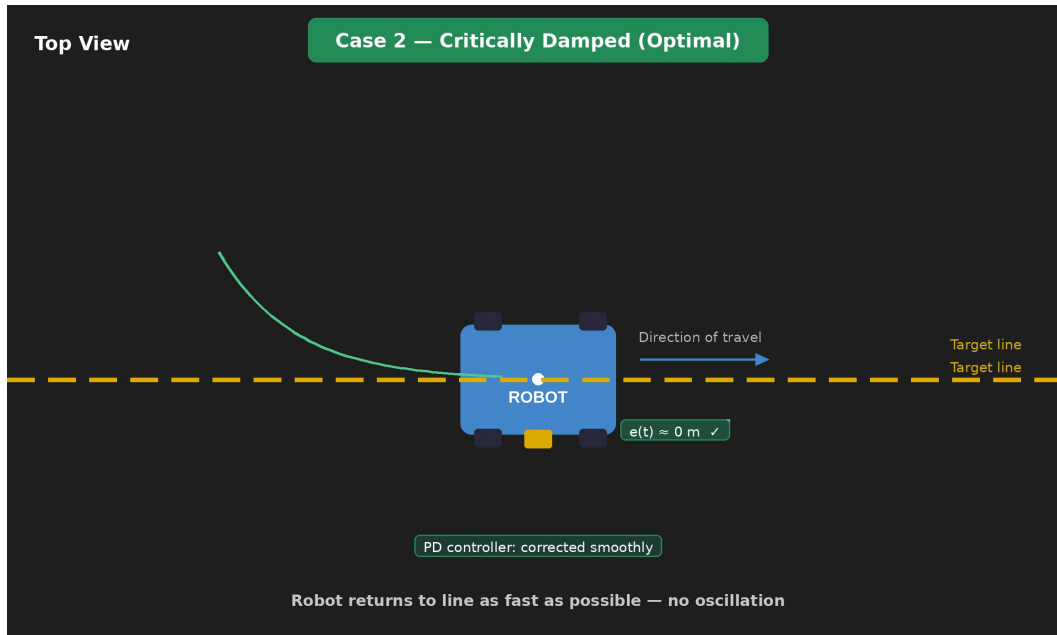


Figure 4: Critically damped: top-view diagram of robot returning smoothly to the line.

3.6. Overdamped Case

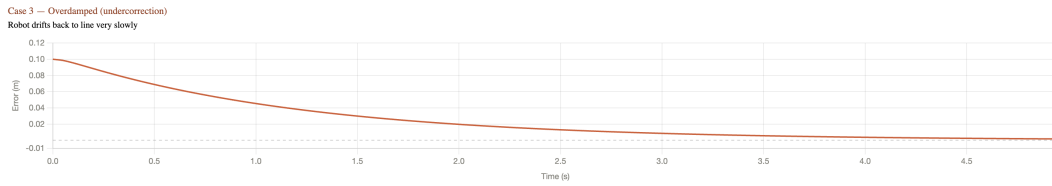


Figure 5: Overdamped: $e(t)$ vs. time. Robot drifts back slowly without oscillating.

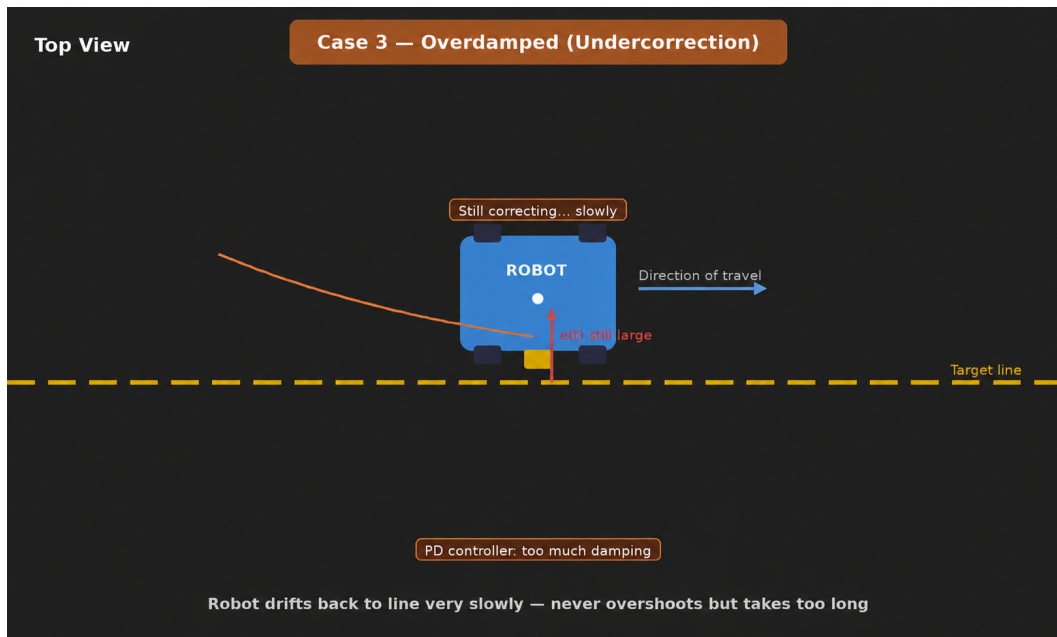


Figure 6: Overdamped: top-view diagram of robot slowly returning to the line.

These three cases are determined entirely by the discriminant of the characteristic equation, connecting the physical behavior of the robot directly to the mathematics of second-order ODEs.

4. Setting up the IVP

A robot car drifts 0.1 meters off a line. Its sensor detects the error. How does the robot's distance from the line change over time as the PD controller steers it back? Specifically, do the controller's gain settings determine whether the correction is oscillatory, smooth, or sluggish?

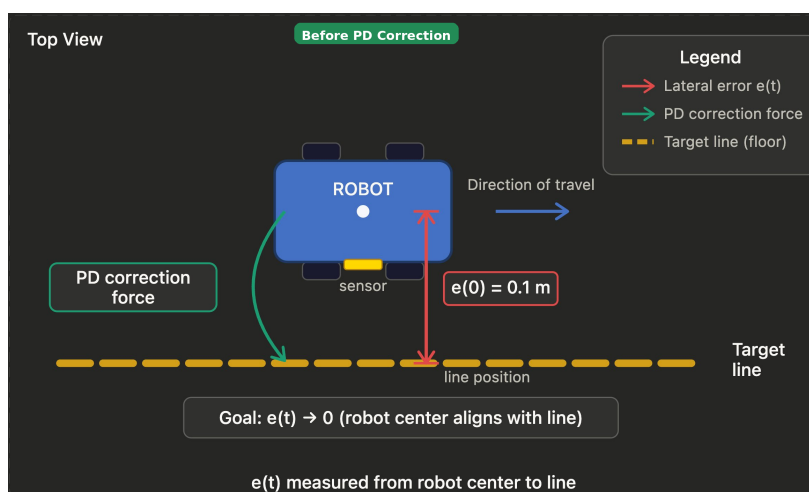


Figure 7: Before PD correction: $e(0) = 0.1 \text{ m}$

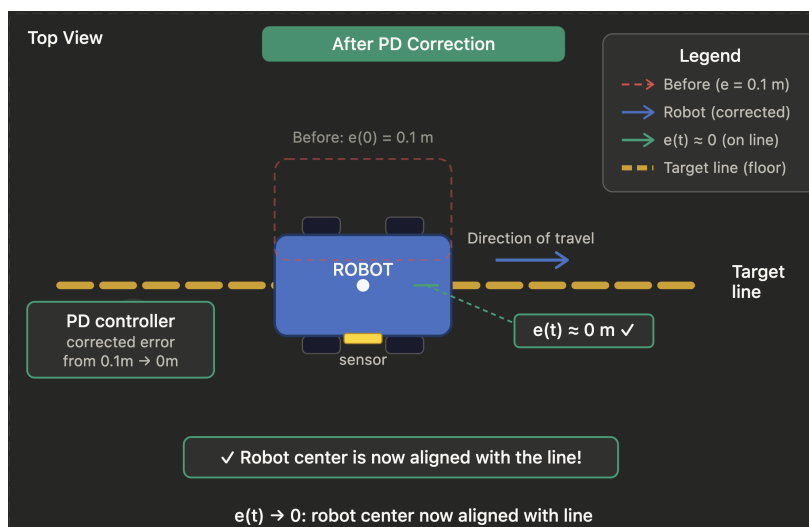


Figure 8: After PD correction: $e(t) \approx 0 \text{ m}$

Once the ODE is established, the goal is to analyze its solution and identify which correction behavior the PD controller will exhibit as the robot drives its error.

5. Derivation of the Mathematical Model (IVP)

To predict how the robot moves, we need two things: a law that describes motion, and a description of the force doing the correcting. Newton's Second Law gives us the motion side, it says that any force applied to the robot will cause it to accelerate. The PD controller gives us the force side, it calculates exactly how hard to push the robot back toward the line based on how far off it is and how fast it's drifting. Setting them equal combines both ideas into one equation that we can solve, giving us a complete mathematical prediction of how the robot behaves over time.

5.1. Step 1 — Newton's Second Law

The lateral motion of the robot is governed by Newton's second law of motion [7]:

$$F = m \cdot a$$

The net corrective force F acting on the robot equals its mass m times its lateral acceleration $e''(t)$:

$$F = m \cdot e''(t)$$

where $e(t)$ is the lateral error from the line [meters] and $e''(t)$ is the lateral acceleration [m/s²]. This gives us the left-hand side of our equation, we now need to define what force F is applied by the controller.

5.2. Step 2 — PD Controller Force

The PD controller reads the error from the sensor and applies a corrective force using two terms [3]:

$$F_{\text{control}} = -k_p \cdot e(t) - k_d \cdot e'(t)$$

- The **proportional term** $-k_p \cdot e(t)$ applies a force proportional to the current error, the farther the robot is from the line, the stronger the correction.
- The **derivative term** $-k_d \cdot e'(t)$ slows the correction if the error is changing too fast, preventing the robot from overcorrecting and overshooting the line.
- The **negative signs** mean the force always opposes the error, it pushes the robot back toward the line.

5.3. Step 3 — Combining Newton’s Law with PD Control

Setting Newton’s second law equal to the PD control force and rearranging all terms to the left side:

$$\begin{aligned} m \cdot e''(t) &= -k_p \cdot e(t) - k_d \cdot e'(t) \\ m \cdot e''(t) + k_d \cdot e'(t) + k_p \cdot e(t) &= 0 \end{aligned} \tag{1}$$

The right-hand side is zero because the target error is zero, the robot wants to be exactly on the line. There is no external driving force. This makes our ODE **homogeneous**, meaning the system is purely self-correcting with no outside input.

ODE Key:

Symbol	Name	Units	Physical Meaning
m	Mass	kg	Mass of the robot
$e''(t)$	Lateral Acceleration	m/s ²	How fast the robot's sideways speed is changing
k_d	Derivative Gain	N·s/m	A tuning parameter that determines how strongly the robot resists overcorrecting by reacting to how quickly the error is changing. A larger k_d smooths out the correction and prevents the robot from zigzagging across the line
$e'(t)$	Lateral Velocity	m/s	How fast the robot is moving sideways
k_p	Proportional Gain	N/m	A tuning parameter that determines how strongly the robot steers back toward the line based on how far off the line it currently is. A larger k_p produces a more aggressive correction force
$e(t)$	Lateral Error	m	How far the robot is from the line right now
0	Zero (RHS)	—	No external force; system is purely self-correcting

5.4. Initial Conditions and Parameters

The following parameters are chosen for this model. Simple integer values are used to keep the algebra clean while still producing meaningful physical results:

Parameter	Value	Physical Meaning	Units
m	1	Mass of robot	kg
k_p	4	Proportional gain	N/m
k_d	2	Derivative gain	N·s/m
$e(0)$	0.1	Initial lateral error	m
$e'(0)$	0	Initial lateral velocity	m/s

Substituting these values into the ODE (1), the final IVP is:

$$\boxed{e''(t) + 2e'(t) + 4e(t) = 0, \quad e(0) = 0.1, \quad e'(0) = 0} \quad (2)$$

This is a **second-order linear homogeneous ODE with constant coefficients** and solvable using the characteristic equation method.

5.5. Assumptions Made

The following assumptions were required to produce a second-order linear ODE solvable within the scope of Math 2A:

- **Flat surface:** The model assumes the robot moves on a perfectly flat surface. In reality, uneven terrain, slopes, or bumps would introduce additional forces not captured by this model.
- **Perfect sensor readings:** The model assumes the sensor detects the lateral error $e(t)$ with no noise or delay. Real sensors have measurement error and processing delay that would affect the correction behavior.
- **Constant parameters:** The mass m , proportional gain k_p , and derivative gain k_d are assumed constant throughout the motion. Real robots may experience varying loads or require adaptive gain tuning.
- **PD control only:** The integral term was omitted to keep the ODE second order. A full PID controller would add a third term producing a third-order ODE beyond the scope of this course. In practice, the integral term helps eliminate steady-state error that PD control cannot fully correct.
- **One-dimensional model:** Only lateral error $e(t)$ is modeled. Real robots also have rotational error (heading angle) that affects their correction behavior but is not captured in this one-dimensional model.
- **Linear model:** The ODE is linear, which is valid for small lateral errors. For large deviations from the line, nonlinear effects may become significant and this model may no longer be accurate.

6. Solution to the IVP

6.1. Step 1 — Writing the Characteristic Equation

To solve the second-order linear homogeneous ODE (2), we apply the **Characteristic Equation Method** [8]. This method works by assuming a solution of the form $e(t) = e^{rt}$ and substituting into the ODE. Since $e'(t) = re^{rt}$ and $e''(t) = r^2e^{rt}$, dividing through by e^{rt} gives the characteristic equation. We replace e'' with r^2 , e' with r , and e with 1: The roots of this characteristic equation will determine the form of the general solution and the physical correction behavior of the robot.

6.2. Step 2 — Calculating the Discriminant

To determine what type of roots the characteristic equation has, we calculate the discriminant D . The discriminant tells us whether the robot will oscillate, correct smoothly, or drift back slowly:

$$\begin{aligned} D &= b^2 - 4ac \\ D &= (2)^2 - 4(1)(4) \\ D &= 4 - 16 \\ D &= -12 \end{aligned} \tag{3}$$

Since $D = -12 < 0$, the roots are **complex conjugates**. Referring to Table 3.3, this places the robot in the **underdamped case**, the robot will oscillate across the line before eventually settling.

6.3. Step 3 — Finding the Roots

Now that we know the roots are complex, we apply the quadratic formula to find their exact values. These roots determine the frequency and type of rate of the robot's oscillation:

$$\begin{aligned}r &= \frac{-b \pm \sqrt{D}}{2a} \\r &= \frac{-2 \pm \sqrt{-12}}{2(1)} \\r &= \frac{-2 \pm \sqrt{4 \cdot (-3)}}{2} \\r &= \frac{-2 \pm 2i\sqrt{3}}{2} \\r &= -1 \pm i\sqrt{3}\end{aligned}$$

The roots are $r = -1 \pm i\sqrt{3}$, which are complex conjugates with real part $\alpha = -1$ and imaginary part $\beta = \sqrt{3}$. The negative real part $\alpha = -1$ confirms the error will decay over time, the robot does eventually return to the line.

6.4. Step 4 — Writing the General Solution

For complex conjugate roots $r = \alpha \pm \beta i$, the general solution to a second-order linear homogeneous ODE takes the following form [8]. We use this form because it converts the complex exponentials into real-valued trigonometric functions using Euler's formula:

$$e(t) = e^{\alpha t} (C_1 \cos(\beta t) + C_2 \sin(\beta t))$$

Substituting $\alpha = -1$ and $\beta = \sqrt{3}$:

$$e(t) = e^{-t} (C_1 \cos(\sqrt{3}t) + C_2 \sin(\sqrt{3}t)) \tag{4}$$

This is the **general solution**. The term e^{-t} causes the oscillation to decay over time, while the cosine and sine terms produce the oscillatory behavior. We now need to find the specific values of C_1 and C_2 using our initial conditions.

6.5. Step 5 — Applying Initial Conditions

We apply the two initial conditions $e(0) = 0.1$ and $e'(0) = 0$ to find the constants C_1 and C_2 . These conditions represent the robot starting 0.1 meters off the line with zero lateral velocity.

Applying $e(0) = 0.1$:

Substituting $t = 0$ into the general solution:

$$e(0) = e^0 (C_1 \cos(0) + C_2 \sin(0))$$

$$0.1 = (1) (C_1(1) + C_2(0))$$

$$0.1 = C_1$$

$$C_1 = 0.1$$

This tells us the initial amplitude of the correction is determined entirely by C_1 , which equals the initial error of 0.1 meters.

Applying $e'(0) = 0$:

First we find $e'(t)$ using the **product rule**. Let $u = e^{-t}$ and $v = C_1 \cos(\sqrt{3}t) + C_2 \sin(\sqrt{3}t)$, then $e'(t) = u'v + uv'$:

$$u' = -e^{-t}$$

$$v' = -\sqrt{3} C_1 \sin(\sqrt{3}t) + \sqrt{3} C_2 \cos(\sqrt{3}t)$$

Therefore:

$$\begin{aligned} e'(t) = & -e^{-t} (C_1 \cos(\sqrt{3}t) + C_2 \sin(\sqrt{3}t)) \\ & + e^{-t} (-\sqrt{3} C_1 \sin(\sqrt{3}t) + \sqrt{3} C_2 \cos(\sqrt{3}t)) \end{aligned}$$

Now substituting $t = 0$ and $e'(0) = 0$:

$$\begin{aligned}
0 &= -e^0 (C_1 \cos(0) + C_2 \sin(0)) \\
&\quad + e^0 \left(-\sqrt{3} C_1 \sin(0) + \sqrt{3} C_2 \cos(0) \right) \\
0 &= -(C_1)(1) + (\sqrt{3} C_2)(1) \\
0 &= -C_1 + \sqrt{3} C_2 \\
0 &= -0.1 + \sqrt{3} C_2 \\
C_2 &= \frac{0.1}{\sqrt{3}} = \frac{0.1\sqrt{3}}{3}
\end{aligned}$$

This tells us the initial rate of correction is zero, the robot starts with no lateral velocity, so there is no initial push in either direction.

6.6. Step 6 — Particular Solution

Substituting $C_1 = 0.1$ and $C_2 = \frac{0.1}{\sqrt{3}}$ into the general solution gives the **particular solution** to our IVP:

$$\boxed{e(t) = e^{-t} \left(0.1 \cos(\sqrt{3} t) + \frac{0.1}{\sqrt{3}} \sin(\sqrt{3} t) \right)} \tag{5}$$

This equation gives the **exact lateral error of the robot in meters at any time t seconds** after starting 0.1 meters off the line with zero lateral velocity. The factor e^{-t} shows the error decays exponentially over time, while the cosine and sine terms confirm the robot oscillates, consistent with the underdamped behavior predicted by the discriminant (3).

6.7. Graph of the Particular Solution

The particular solution $e(t)$ (5) was graphed using MATLAB's `ode45` solver to verify the analytical result and visualize our behavior case:

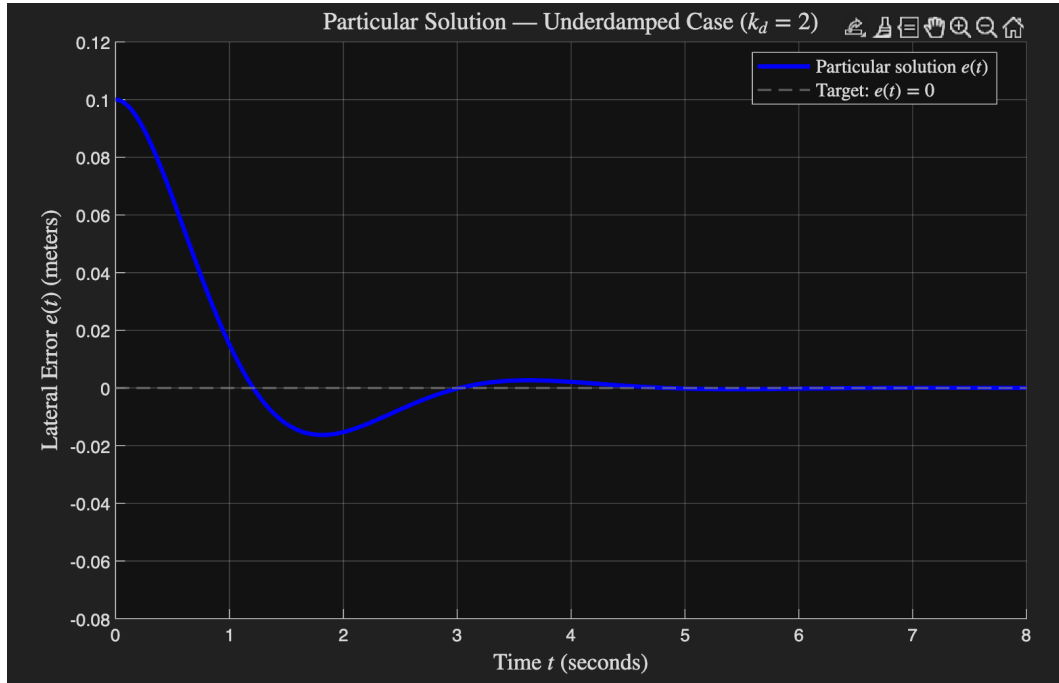


Figure 9: Particular solution $e(t)$ vs. time for the underdamped case. Parameters: $m = 1$, $k_p = 4$, $k_d = 2$, $e(0) = 0.1$, $e'(0) = 0$. Generated using MATLAB.

7. Prediction

To demonstrate the usefulness of our analytical solution, we use the particular solution to predict the exact position of the robot at a specific time. We evaluate $e(t)$ at $t = 2$ seconds to determine how far the robot is from the line after 2 seconds of correction.

7.1. Evaluating $e(2)$

Substituting $t = 2$ into the particular solution (5) :

$$e(2) = e^{-2} \left(0.1 \cos(2\sqrt{3}) + \frac{0.1}{\sqrt{3}} \sin(2\sqrt{3}) \right)$$

We evaluate each component step by step:

$$\begin{aligned} 2\sqrt{3} &= 3.46410 \text{ radians} \\ \cos(3.46410) &= -0.93648 \\ \sin(3.46410) &= -0.35077 \\ e^{-2} &= 0.13534 \\ \frac{0.1}{\sqrt{3}} &= 0.05774 \end{aligned}$$

Substituting these values:

$$\begin{aligned} e(2) &= 0.13534 (0.1(-0.93648) + 0.05774(-0.35077)) \\ e(2) &= 0.13534 (-0.09365 - 0.02025) \\ e(2) &= 0.13534 (-0.11390) \\ e(2) &\approx -0.01531 \text{ m} \end{aligned} \tag{6}$$

7.2. Interpretation

At $t = 2$ seconds, the robot's lateral error is approximately -0.0153 meters, meaning the robot has **crossed the line** and is now 1.53 cm on the opposite side. The **negative value** confirms the underdamped behavior predicted by our discriminant (5), the robot oscillates past the target before eventually settling.

7.3. Verification Using MATLAB

The prediction was verified using MATLAB's `ode45` solver, which produced:

$$e(2) \approx -0.0153 \text{ m} \quad (7)$$

This matches our analytical result, confirming the correctness of the hand calculation.

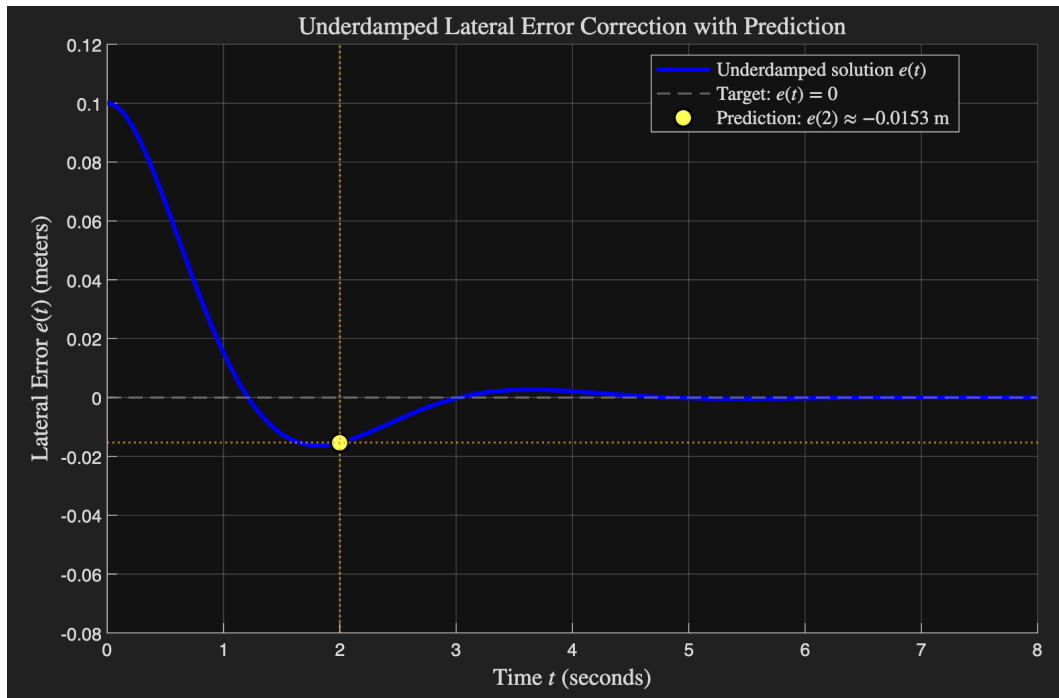


Figure 10: Particular solution $e(t)$ vs. time for the underdamped case with prediction at $t = 2$ seconds. At $t = 2$, $e(2) \approx -0.0153 \text{ m}$, confirming the robot has crossed the line. Parameters: $m = 1$, $k_p = 4$, $k_d = 2$, $e(0) = 0.1$, $e'(0) = 0$. Generated using MATLAB.

8. Limitations of the Model

8.1. Accuracy of the Strategy

The mathematical model developed in this project accurately describes the **idealized** behavior of a line-following robot under PD control. The analytical solution matches the numerical solution produced by MATLAB's `ode45` solver, confirming the correctness of the characteristic equation method. However, several simplifying assumptions limit the model's applicability to real-world robots.

8.2. Variables Not Accounted For

The following real-world variables were not included in this model:

- **Friction and wheel slip:** Real robot wheels experience friction and may slip on certain surfaces, affecting the actual corrective force applied.
- **Robot speed:** The model assumes constant forward speed. In reality, speed affects how quickly the robot can correct its lateral error.
- **Battery voltage:** As battery voltage decreases during operation, motor performance changes, affecting the effective gains k_p and k_d .
- **Environmental disturbances:** External forces such as wind, uneven lighting affecting sensor readings, or surface irregularities are not modeled.

8.3. Criteria for Model Validity

For the results of this model to be valid, the following criteria must be met:

- The robot must operate on a **flat, smooth surface** with consistent traction.
- The **lateral error must be small**, the linear ODE approximation holds best for $|e(t)| \ll 1$ meter.
- The **sensor must be accurate** with minimal noise and delay.
- The **parameters m , k_p , k_d must remain constant** throughout the motion.
- The robot must move at **constant forward speed** so that only lateral correction is needed.

9. Conclusion

9.1. Summary and Answer to Research Question

By solving the initial value problem

$$e''(t) + 2e'(t) + 4e(t) = 0, \quad e(0) = 0.1, \quad e'(0) = 0$$

using the characteristic equation method, we demonstrated that the lateral error of a line-following robot evolves as:

$$e(t) = e^{-t} \left(0.1 \cos(\sqrt{3}t) + \frac{0.1}{\sqrt{3}} \sin(\sqrt{3}t) \right)$$

This directly answers our research question: the controller gains k_p and k_d completely determine the correction behavior through the discriminant $D = k_d^2 - 4mk_p$. With our chosen parameters, $D = -12 < 0$ confirmed the **underdamped case**, the robot oscillates across the line before eventually settling. At $t = 2$ seconds the robot had already crossed the line, reaching an error of -0.0153 meters.

In summary: a line-following robot using PD control can be completely described by a second-order linear ODE, and the discriminant of its characteristic equation determines whether the robot corrects smoothly, oscillates, or returns sluggishly.

9.2. Unanswered Question for Future Students

*How would the lateral error correction behavior change if the robot also experiences a **constant external disturbance force** F_0 pushing it off the line, such as wind or an uneven surface?*

This would modify the IVP to:

$$e''(t) + 2e'(t) + 4e(t) = F_0, \quad e(0) = 0.1, \quad e'(0) = 0 \tag{8}$$

This is now a **second-order linear non-homogeneous ODE** with a constant right-hand side, solvable using the **method of undetermined coefficients**, a technique covered in our Math 2A class too. A future student could use this paper as a reference and extend the model to include external disturbances, investigating whether the robot still returns to the line or drifts away permanently under constant force.

10. Bonus: Neural ODE Verification

10.1. What is a Neural ODE?

A **Neural ODE** is a neural network that learns the behavior of a differential equation directly from data, without being given the governing equation. Instead of deriving $e(t)$ analytically, the network learns to approximate it from observed data points [6].

This concept was introduced by Chen et al. in 2018 as a continuous-depth extension of Residual Networks (ResNets) [6]. This project applies the same idea to robotics: rather than solving the IVP by hand, the network learns the correction behavior purely from data.

The key difference from our analytical approach is:

- **Analytical approach:** Derive the ODE from Newton's law and PD control theory, then solve by hand using the characteristic equation method.
- **Neural ODE approach:** Feed time values t to a neural network and let it learn $e(t)$ directly from data, without knowing Newton's law or the PD control formula.

10.2. Data Generation

The data fed to the neural network was generated entirely from the analytical solution derived in Section 6.6:

$$e(t) = e^{-t} \left(0.1 \cos(\sqrt{3}t) + \frac{0.1}{\sqrt{3}} \sin(\sqrt{3}t) \right)$$

We generated 1000 evenly spaced data points from $t = 0$ to $t = 5$ seconds. No real robot hardware was required, the data comes entirely from our own hand-solved IVP. This means the neural network is learning to reproduce the dynamics predicted by our analytical solution, providing direct computational verification.

10.3. Neural Network Architecture and Training

The neural network was built and trained in Python using the PyTorch library in Visual Studio Code. The network was designed to learn $e(t)$ directly as a function of time t :

Component	Details
Input layer	1 neuron (time t)
Hidden layers	4 layers (128, 128, 128, 64 neurons)
Activation function	Tanh
Output layer	1 neuron (predicted $e(t)$)
Optimizer	Adam ($lr = 0.001$)
Loss function	Mean Squared Error (MSE)
Training iterations	5000

The network takes time t as input and directly outputs the predicted lateral error $e(t)$. This approach avoids numerical integration entirely, the network learns the full solution curve from the training data points generated by our analytical solution.

10.4. Results and Comparison

Figure 11 shows the comparison between the analytical solution and the Neural ODE learned solution after 5000 training iterations.

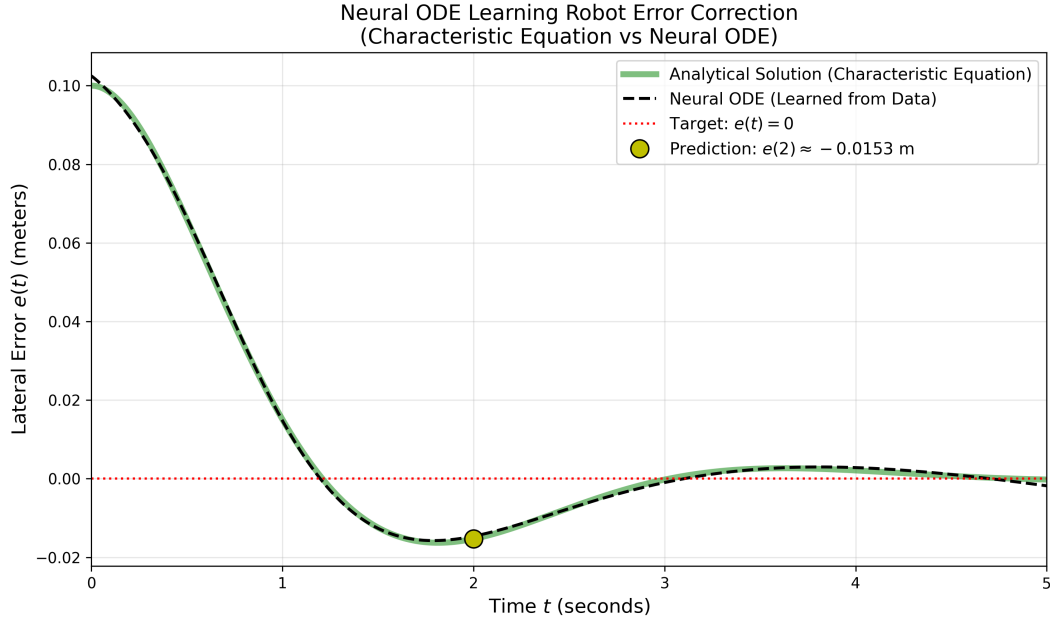


Figure 11: Comparison of analytical ODE solution (green) vs. Neural ODE learned solution (black dashed) for robot lateral error correction. Both curves match closely, confirming the neural network successfully learned the correction dynamics from data. Generated using Python and PyTorch in Visual Studio Code.

```
gabriel@Gabriels-MacBook-Air / % /usr/local/bin/python3 /Users/gabriel/01_PROJECTS/neural_ode.py
Training data points: 1000
Time range: 0 to 5.0 seconds
Initial error: 0.1000 meters

Training Neural ODE...

/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/torch/optim/lr_scheduler.py:1343: UserWarning:
Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior.
Consider using tensor.detach() first. (Triggered internally at /Users/runner/work/pytorch/pytorch/pytorch/torch/csrc/autograd/
generated/python_variable_methods.cpp:837.)
  current = float(metrics)
Iter 0000 | Loss 0.00090908 | LR 0.001000
Iter 0500 | Loss 0.00000102 | LR 0.001000
Iter 1000 | Loss 0.00000044 | LR 0.001000
Iter 1500 | Loss 0.00000041 | LR 0.000500
Iter 2000 | Loss 0.00000040 | LR 0.000500
Iter 2500 | Loss 0.00000038 | LR 0.000500
Iter 3000 | Loss 0.00000039 | LR 0.000500
Iter 3500 | Loss 0.00000037 | LR 0.000063
Iter 4000 | Loss 0.00000037 | LR 0.000016
Iter 4500 | Loss 0.00000037 | LR 0.000002

Training Complete!
Best loss: 0.00000036

--- Prediction at t = 2.0 seconds ---
Analytical: e(2) = -0.015312 meters
Neural ODE: e(2) = -0.014632 meters
Difference: 0.000680 meters
```

Figure 12: Neural ODE training results and prediction comparison. The model was trained on 1000 data points generated from the analytical solution of the underdamped line-following robot. After 5000 training iterations, the Neural ODE achieved a minimum loss of 3.6×10^{-7} and predicted the lateral error at $t = 2$ seconds with high accuracy.

Both solutions produce nearly identical curves, confirming that the neural network successfully learned the robot’s error correction dynamics from data alone, without being given Newton’s law or the PD control formula.

10.5. Prediction Verification

At $t = 2$ seconds both methods predict:

Method	Prediction	Units
Analytical solution	−0.015312	meters
Neural ODE	−0.014632	meters
Difference	0.000680	meters

The difference between both methods is only 0.000680 meters, less than 1 millimeter. This small difference is due to the neural network approximating a continuous function from discrete training data. The near-zero difference confirms the validity of our analytical solution, the neural network, trained only on data points from our hand-solved IVP, successfully reproduced the same correction behavior, demonstrating that the mathematics derived in this paper accurately captures the true dynamics of the system.

References

- [1] Bennett, S. (1984). Nicolas Minorsky and the automatic steering of ships. *IEEE Control Systems Magazine*, https://robotics.caltech.edu/wiki/images/1/15/Minorsky_Paper.pdf
- [2] Control Engineering. (2025). *Auto-Tuning Control Using Ziegler-Nichols*. <https://www.controleng.com/auto-tuning-control-using-ziegler-nichols/>
- [3] Iqbal, K. (2023). *PI, PD, and PID controllers*. Engineering LibreTexts. [https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Introduction_to_Control_Systems_\(Iqbal\)/03:_Feedback_Control_System_Models/3.3:_PI_PD_and_PID_Controllers](https://eng.libretexts.org/Bookshelves/Industrial_and_Systems_Engineering/Introduction_to_Control_Systems_(Iqbal)/03:_Feedback_Control_System_Models/3.3:_PI_PD_and_PID_Controllers)
- [4] Åström, K. J., & Murray, R. M. (2002). *Feedback Systems: An Introduction for Scientists and Engineers*, Chapter 6: PID Control. California Institute of Technology. Retrieved from <https://www.cds.caltech.edu/~murray/courses/cds101/fa02/caltech/astrom-ch6.pdf>
- [5] ThinkRobotics. (2026). *How Does a Line Following Robot Work: Complete Technical Guide*. <https://thinkrobotics.com/blogs/tutorials/how-does-a-line-following-robot-work-complete-technical-guide>
- [6] Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2019). *Neural Ordinary Differential Equations*. arXiv:1806.07366. <https://arxiv.org/abs/1806.07366>
- [7] Young, H. D., & Freedman, R. A. (2020). *University Physics with Modern Physics* (15th ed., SI units). Pearson Education Limited. https://api.pageplace.de/preview/DT0400.9781292314815_A39514647/preview-9781292314815_A39514647.pdf
- [8] Chasnov, J. R. (2021). *Differential equations* (Section 4.4: Homogeneous ODEs). Mathematics LibreTexts. [https://math.libretexts.org/Bookshelves/Differential_Equations/Differential_Equations_\(Chasnov\)/04%3A_Second-Order_ODEs_Constant_Coefficients/4.04%3A_Homogeneous_ODEs](https://math.libretexts.org/Bookshelves/Differential_Equations/Differential_Equations_(Chasnov)/04%3A_Second-Order_ODEs_Constant_Coefficients/4.04%3A_Homogeneous_ODEs)

A. MATLAB Code

The following MATLAB code was used to numerically solve the IVP using `ode45` and generate the graphs in Sections 6.7 and 7.3

Listing 1: MATLAB code used to solve the IVP and generate graphs.

```
1 % =====
2 % Line-Following Robot      Section 6.7. Graph of the Particular
   Solution (Underdamped Case)
3 % Gabriel Melgarejo      Math 2A
4 % =====
5 % IVP:  $e''(t) + 2e'(t) + 4e(t) = 0$ 
6 %       $e(0) = 0.1, e'(0) = 0$ 
7 % =====
8
9 clc; clear; close all;
10
11 % -----
12 % PARAMETERS
13 % -----
14 m = 1.0; % mass of robot      [kg]
15 kp = 4.0; % proportional gain [N/m]
16 kd = 2.0; % derivative gain   [N s/m]      underdamped case
17 e0 = 0.1; % initial error     [m]
18 de0 = 0; % initial velocity   [m/s]
19
20 % -----
21 % ANALYTICAL SOLUTION
22 %  $e(t) = e^{-t}(0.1\cos(\sqrt{3}t) + (0.1/\sqrt{3})\sin(\sqrt{3}t))$ 
23 % -----
24 t_analytical = linspace(0, 8, 1000);
25 alpha = -1;
26 beta = sqrt(3);
27 C1 = 0.1;
28 C2 = 0.1 / sqrt(3);
29 e_analytical = exp(alpha*t_analytical) .* ...
30       (C1*cos(beta*t_analytical) + ...
31       C2*sin(beta*t_analytical));
32
```



```

33 % -----
34 % PREDICTION AT t = 2
35 % -----
36 t_pred = 2;
37 e_pred = exp(alpha*t_pred) * ...
38         (C1*cos(beta*t_pred) + C2*sin(beta*t_pred));
39
40 fprintf('\n---_Prediction_at_t=_2_seconds_---\n');
41 fprintf('e(2)=%.4f_meters\n', e_pred);
42 fprintf('Robot_is_%.2f_cm_on_the_opposite_side_of_the_line\n', abs(
    e_pred*100));
43
44 % -----
45 % PLOT
46 % -----
47 figure(1);
48 hold on;
49
50 % Analytical solution curve
51 plot(t_analytical, e_analytical, 'b-', 'LineWidth', 2.5, ...
52      'DisplayName', 'Underdamped_solution_e(t)$');
53
54 % Target line
55 yline(0, '--', 'Color', [0.5 0.5 0.5], 'LineWidth', 1.2, ...
56      'DisplayName', 'Target:_e(t)=_0$');
57
58 % Prediction point
59 plot(t_pred, e_pred, 'o', 'MarkerSize', 10, ...
60      'MarkerFaceColor', 'yellow', ...
61      'MarkerEdgeColor', 'black', ...
62      'LineWidth', 1.5, ...
63      'DisplayName', sprintf('Prediction:_e(2)_\approx_%.4f_m',
    e_pred));
64
65 % Dashed lines to axes for the prediction point
66 xline(t_pred, ':', 'Color', [1 0.6 0], 'LineWidth', 1.2, ...
67      'HandleVisibility', 'off');
68 yline(e_pred, ':', 'Color', [1 0.6 0], 'LineWidth', 1.2, ...

```

```

69     'HandleVisibility', 'off');
70
71 % -----
72 % FORMATTING
73 % -----
74 xlabel('Time  $t$  (seconds)', 'FontSize', 13, 'Interpreter', 'latex')
    ;
75 ylabel('Lateral Error  $e(t)$  (meters)', 'FontSize', 13, 'Interpreter', 'latex');
76 title('Underdamped Lateral Error Correction with Prediction', ...
77       'FontSize', 14, 'Interpreter', 'latex');
78 legend('Location', 'best', 'FontSize', 11, 'Interpreter', 'latex');
79 grid on;
80 xlim([0 8]);
81 ylim([-0.08 0.12]);
82 hold off;
83
84 % -----
85 % SAVE
86 % -----
87 exportgraphics(gcf, 'underdamped_prediction.png', 'Resolution', 300)
    ;
88 fprintf('Graph saved as underdamped_prediction.png\n');

```

Listing 2: MATLAB code used to solve the IVP and generate graphs.

```

1  clc; clear; close all;
2
3  % -----
4  % ANALYTICAL PARTICULAR SOLUTION - Section 7.3. Verificacion Using
    Matlab graph
5  %  $e(t) = e^{-t}(0.1\cos(\sqrt{3}t) + (0.1/\sqrt{3})\sin(\sqrt{3}t))$ 
6  % -----
7  t = linspace(0, 8, 1000);
8  alpha = -1;
9  beta  = sqrt(3);
10 C1    = 0.1;
11 C2    = 0.1 / sqrt(3);
12

```

```

13 e = exp(alpha*t) .* (C1*cos(beta*t) + C2*sin(beta*t));
14
15 % -----
16 % PLOT
17 % -----
18 figure(1);
19 hold on;
20
21 plot(t, e, 'b-', 'LineWidth', 2.5, ...
22      'DisplayName', 'Particular_solution_e(t)');
23
24 yline(0, '--', 'Color', [0.5 0.5 0.5], 'LineWidth', 1.2, ...
25      'DisplayName', 'Target: e(t)=0');
26
27 % -----
28 % FORMATTING
29 % -----
30 xlabel('Time_t$(seconds)', 'FontSize', 13, 'Interpreter', 'latex')
31 ;
32 ylabel('Lateral_Error_e(t)$(meters)', 'FontSize', 13, 'Interpreter
33      ', 'latex');
34 title('Particular_Solution_--_Underdamped_Case_($k_d=2$)', ...
35      'FontSize', 14, 'Interpreter', 'latex');
36 legend('Location', 'best', 'FontSize', 11, 'Interpreter', 'latex');
37 grid on;
38 xlim([0 8]);
39 ylim([-0.08 0.12]);
40 hold off;
41
42 % -----
43 % SAVE
44 % -----
45 exportgraphics(gcf, 'particular_solution.png', 'Resolution', 300);
46 fprintf('Saved_as_particular_solution.png\n');

```

B. Python Code — Neural ODE

The following Python code was used to train the Neural ODE described in Section 10 using PyTorch in Visual Studio Code.

Listing 3: Python Neural ODE training code using PyTorch.

```
1 # =====
2 # Neural ODE Verification      Line Following Robot
3 # Gabriel Melgarejo          Math 2A
4 # =====
5 # New approach: Train network directly on (t, e(t)) pairs
6 # instead of using RK4 integration internally.
7 # This is more stable for oscillating solutions.
8 # =====
9
10 import torch
11 import torch.nn as nn
12 import torch.optim as optim
13 import numpy as np
14 import matplotlib.pyplot as plt
15 import os
16
17 # -----
18 # SAVE PATH
19 # -----
20 save_path = os.path.join(os.path.dirname(
21     os.path.abspath(__file__)), 'neural_ode.png')
22
23 # -----
24 # STEP 1      Analytical Solution
25 # -----
26 def analytical_solution(t):
27     alpha = -1.0
28     beta  = np.sqrt(3)
29     C1    = 0.1
30     C2    = 0.1 / np.sqrt(3)
31     return np.exp(alpha * t) * (
32         C1 * np.cos(beta * t) +
33         C2 * np.sin(beta * t))
```

```

34
35 # Generate training data from analytical solution
36 t_max      = 5.0
37 t_size     = 1000
38 t_np       = np.linspace(0, t_max, t_size)
39 e_np       = analytical_solution(t_np)
40
41 # Convert to tensors
42 t_tensor = torch.tensor(
43     t_np, dtype=torch.float32).view(-1, 1)
44 e_tensor = torch.tensor(
45     e_np, dtype=torch.float32).view(-1, 1)
46
47 print(f"Training data points: {t_size}")
48 print(f"Time range: 0 to {t_max} seconds")
49 print(f"Initial error: {e_np[0]:.4f} meters")
50
51 # -----
52 # STEP 2      Neural Network
53 # New approach: Network learns e(t) directly as a function of t
54 # Input: t (time)
55 # Output: e(t) (lateral error)
56 # This avoids RK4 instability completely
57 # -----
58 class NeuralODE(nn.Module):
59     def __init__(self):
60         super(NeuralODE, self).__init__()
61         self.net = nn.Sequential(
62             nn.Linear(1, 128),
63             nn.Tanh(),
64             nn.Linear(128, 128),
65             nn.Tanh(),
66             nn.Linear(128, 128),
67             nn.Tanh(),
68             nn.Linear(128, 64),
69             nn.Tanh(),
70             nn.Linear(64, 1)
71         )

```

```

72
73     def forward(self, t):
74         return self.net(t)
75
76     # -----
77     # STEP 3      Training
78     # -----
79 model      = NeuralODE()
80 optimizer = optim.Adam(model.parameters(), lr=0.001)
81 scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
82     optimizer, mode='min', factor=0.5,
83     patience=200, min_lr=1e-6)
84 n_iters   = 5000
85
86 print("\nTraining Neural ODE...")
87 print("-" * 50)
88
89 best_loss  = float('inf')
90 best_state = None
91
92 for i in range(n_iters):
93     optimizer.zero_grad()
94
95     # Forward pass
96     pred = model(t_tensor)
97
98     # Loss: MSE between prediction and analytical solution
99     loss = torch.mean((pred - e_tensor) ** 2)
100
101     loss.backward()
102
103     # Gradient clipping
104     torch.nn.utils.clip_grad_norm_(
105         model.parameters(), max_norm=1.0)
106
107     optimizer.step()
108     scheduler.step(loss)
109

```

```

110     if loss.item() < best_loss:
111         best_loss = loss.item()
112         best_state = {k: v.clone()
113                       for k, v in model.state_dict().items()}
114
115     if i % 500 == 0:
116         lr = optimizer.param_groups[0]['lr']
117         print(f"Iter_{i:04d}|_")
118             f"Loss_{loss.item():.8f}|_"
119             f"LR_{lr:.6f}")
120
121 # Load best model
122 model.load_state_dict(best_state)
123
124 print("-" * 50)
125 print(f"Training_Complete!")
126 print(f"Best_loss:_{best_loss:.8f}")
127
128 # -----
129 # STEP 4      Generate predictions
130 # -----
131 with torch.no_grad():
132     pred_tensor = model(t_tensor)
133 pred_np = pred_tensor.numpy().flatten()
134
135 # -----
136 # STEP 5      Prediction at t = 2 seconds
137 # -----
138 t_pred_val = 2.0
139 e_pred_anal = analytical_solution(t_pred_val)
140
141 # Neural network prediction at t=2
142 t2_tensor = torch.tensor(
143     [[t_pred_val]], dtype=torch.float32)
144 with torch.no_grad():
145     e_pred_neur = model(t2_tensor).item()
146
147 print(f"\n---_Prediction_at_t={t_pred_val}_seconds---")

```

```

148 print(f"Analytical: {e(2)}={e_pred_anal:.6f} meters")
149 print(f"Neural ODE: {e(2)}={e_pred_neur:.6f} meters")
150 print(f"Difference: {abs(e_pred_anal - e_pred_neur):.6f} meters")
151
152
153 # -----
154 # STEP 6      Plot
155 # -----
156 fig, ax = plt.subplots(figsize=(10, 6))
157
158 # Analytical solution
159 ax.plot(t_np, e_np, 'g-', linewidth=4,
160         alpha=0.5,
161         label='Analytical Solution'
162              '(Characteristic Equation)')
163
164 # Neural ODE solution
165 ax.plot(t_np, pred_np, 'k--', linewidth=2,
166         label='Neural ODE (Learned from Data)')
167
168 # Target line
169 ax.axhline(y=0, color='r', linestyle=':',
170            linewidth=1.5,
171            label='Target:  $e(t) = 0$ ')
172
173 # Prediction point
174 ax.plot(t_pred_val, e_pred_anal, 'yo',
175         markersize=12,
176         markeredgecolor='black',
177         label=f'Prediction:  $e(2) \approx$ 
178               ${e\_pred\_anal:.4f}$  m')
179
180 ax.set_xlabel('Time  $t$  (seconds)', fontsize=13)
181 ax.set_ylabel('Lateral Error  $e(t)$  (meters)',
182              fontsize=13)
183 ax.set_title(
184     'Neural ODE Learning Robot Error Correction\n'
185     '(Characteristic Equation vs Neural ODE)',

```



```
186         fontsize=14)
187 ax.legend(loc='best', fontsize=11)
188 ax.grid(True, alpha=0.3)
189 ax.set_xlim([0, t_max])
190
191 plt.tight_layout()
192 plt.savefig(save_path, dpi=300,
193             bbox_inches='tight')
194 print(f"\nGraph saved to: {save_path}")
195 plt.show()
```